

MEMORIA DE PRÁCTICAS

3^o Programación
Dinámica

PROGRAMACIÓN DINÁMICA (SCMC)

El problema SCMC, para dos cadenas r y t , puede ser resuelto usando programación dinámica. Para ello, observa que:

- 1) Si el último símbolo de las dos cadenas coincide, la supersecuencia común más corta de ambas debe acabar necesariamente con dicho símbolo.
- 2) En otro caso, la supersecuencia común más corta podrá finalizar bien con el último símbolo de la primera cadena, bien con el último símbolo de la segunda. De ambas opciones, tomaremos la que dé lugar a una supersecuencia más corta.

OBJETIVOS DE LA PRÁCTICA:

- a) Plantear la relación de recurrencias correspondiente al problema SCMC.
- b) Implementar una solución por programación dinámica a este problema con complejidad $O(|r| \times |t|)$. Para ello, se rellenará una matriz $m[0..|r|][0..|t|]$ usando la relación de recurrencias del punto a), de modo que la entrada $m[i][j]$ (para $0 \leq i \leq |r|$ y $0 \leq j \leq |t|$) indique la longitud de la SCMC de las cadenas $r_1, r_2 \dots r_i$ y $t_1, t_2 \dots t_j$.
- c) Implementar el algoritmo que reconstruya, usando la tabla del punto b), una supersecuencia común más corta (nótese que, aunque pueden existir varias soluciones óptimas para una instancia, estaremos interesados en obtener solo una).
- d) Evaluar experimentalmente la corrección de su implementación, tal como se explica posteriormente.

Para resolver este problema, definiremos una clase SCMC, de modo que un objeto instancia de la clase representa una instancia del problema (dada por un alfabeto sigma y dos cadenas r y t).

La clase SCMC define dos constructores (ya implementados en el esqueleto proporcionado). Los métodos $r()$, $t()$ y $\text{sigma}()$ son selectores que permiten acceder a las distintas componentes de una instancia. Se suministra, ya implementado, el método $\text{unaSoluciónFB}()$, que devuelve una solución al problema usando el método de fuerza bruta descrito posteriormente. La variable de instancia m es la matriz que se usará para resolver el problema por programación dinámica.

El método $\text{solucionaPD}()$ debe solucionar la instancia aplicando programación dinámica. Para ello, rellenará la matriz m (apartado b) de la práctica). Una vez resuelta una instancia, deberá poderse obtener la longitud de la supersecuencia común más corta al invocar el método $\text{longitudDeSoluciónPD}()$. El método $\text{unaSoluciónPD}()$ debe devolver una supersecuencia común más corta para la instancia (apartado c) de la práctica), usando para ello la información almacenada en la tabla m .

<<Java Class>>	
 SCMC	
(default package)	
▣ r: String ▣ t: String ▣ sigma: String ▣ m: int[][] ▣ rnd: Random	
⚡ SCMC(String,String,String) ⚡ SCMC(int,int) ● $r()$:String ● $t()$:String ● $\text{sigma}()$:String ● $m(\text{int},\text{int})$:int ● $\text{solucionaPD}()$:void ● $\text{longitudDeSoluciónPD}()$:int ● $\text{unaSoluciónPD}()$:String ● $\text{toString}()$:String ● $\text{unaSoluciónFB}()$:String ■ $\text{unaSoluciónFB}(\text{String},\text{int})$:String	

Para la resolución de este ejercicio, a parte de la clase `SCMC.java` ya nombrada y explicada, se nos proporcionan las clases `TestsCorreccion.java` y `TestsTiempos.java`. Ambos con sus respectivas clases y archivos, junto a la clase `Utils.java` de la cual usaremos métodos auxiliares para mejorar la comprensión de la solución que también explicaremos.

En primer lugar, como en todo ejercicio de programación dinámica, deberemos realizar un planteamiento de la solución mediante una secuencia de decisiones. Esto es algo que se nos da en el enunciado por lo que nosotros debemos ver que dicho camino cumple el principio de optimalidad y plantear su ecuación de Bellman.

Por reducción al absurdo, si la elección del siguiente símbolo no es la óptima, entonces existiría un camino más corto el cual comenzaría escogiendo el símbolo contrario.

Debemos plantear una definición recursiva de la solución del problema. En este nos piden encontrar **la cadena de longitud mínima supersecuencia común entre dos cadenas**. Siguiendo los pasos del enunciado y denotando $M(i, j)$ tal que i es la longitud de la cadena r y j la longitud de la cadena t . Además, denotaremos r_i e t_j como el último símbolo de la cadena r y el último símbolo de la cadena t , respectivamente.

$$M(i, j) = \begin{cases} i + j & i = 0 \vee j = 0 \\ M(i - 1, j - 1) + 1 & r_i = t_j \\ \min\{M(i - 1, j), M(i, j - 1)\} + 1 & r_i \neq t_j \end{cases}$$

Ahora, debemos implementar dicha ecuación, definiendo una estructura auxiliar, en nuestro caso una tabla. Por tanto, completaremos el método `solucionaPD()` y después, imprimiremos su solución la cual, una vez rellena la tabla, estará en la posición $M(r.length(), t.length())$. Por tanto, comencemos a comentar nuestro código:

```
public void solucionaPD() {

    int filas = r.length();
    int cols = t.length();

    for (int i = 0; i <= filas; i++) {
        m[i][0] = i;
    }
    for (int j = 0; j <= cols; j++) {
        m[0][j] = j;
    }

    for (int i = 1; i <= filas; i++) {
        for (int j = 1; j <= cols; j++) {
            if (Utils.charEn(r,i)== Utils.charEn(t,j)) {
                m[i][j] = m[i - 1][j - 1] + 1;
            } else {
                m[i][j] = 1 + Integer.min(m[i][j - 1], m[i-1][j]);
            }
        }
    }
}
```

```

public int longitudDeSolucionPD() {
    return m(r.length(), t.length());
}

public String unaSolucionPD() {

    int i = r.length();
    int j = t.length();
    StringBuilder str = new StringBuilder("");

    while (i > 0 && j > 0) {
        if (Utils.charEn(r,i) == Utils.charEn(t,j)) {
            str.append(Utils.charEn(r,i));
            i--;
            j--;
        } else if (m[i - 1][j] < m[i][j - 1]){
            str.append(Utils.charEn(r,i));
            i--;
        } else {
            str.append(Utils.charEn(t,j));
            j--;
        }
    }

    while (j > 0) {
        str.append(Utils.charEn(t,j));
        j--;
    }

    while (i > 0) {
        str.append(Utils.charEn(r,i));
        i--;
    }

    return str.reverse().toString();
}

```

En primer lugar, veamos el método `solucionaPD()`, el cual, con complejidad $O(|r| \times |t|)$, rellena la tabla siguiendo nuestra ecuación planteada. Es claro que los casos base son la primera fila y columna de la matriz en los cuales si una de las cadenas tiene longitud 0, la supersecuencia más corta será la longitud de la cadena contraria. En el caso general,

```

for (int i = 1; i <= filas; i++) {
    for (int j = 1; j <= cols; j++) {
        if (Utils.charEn(r,i) == Utils.charEn(t,j)) {
            m[i][j] = m[i - 1][j - 1] + 1;
        } else {
            m[i][j] = 1 + Integer.min(m[i][j - 1], m[i-1][j]);
        }
    }
}

```

Usamos el método `Utils.charEn(cadena,pos)`, el cuál accede a la última posición de las cadenas, las cuales compararemos para ver si son iguales en su último carácter. Además, usamos el método `Integer.min` para elegir cuál de las dos posiciones tiene la solución mínima.

Es claro que la complejidad es la requerida ya que la implementación son dos bucles anidados de tamaños r y t longitudes de dichas cadenas. Una vez rellena la tabla, para dar la longitud de la supersecuencia común más corta entre dos cadenas, simplemente debemos elegir la posición de la matriz que nos requiera, en nuestro caso, para dos cadenas cualesquiera, deberemos acceder a la posición M (longitud 1º cadena, longitud 2º cadena) gracias al método `longitudDeSolucionPD()`:

```
public int longitudDeSolucionPD() {
    return m(r.length(), t.length());
}
```

Por último, debemos encontrar la supersecuencia pedida mediante el método `unaSolucionPD()`, el cual devolverá la cadena recorriendo la tabla desde abajo hacia arriba. En mi caso, he construido un `StringBuilder` por el cual he ido coleccionando los últimos símbolos de la cadena indicada y realizando `str.reverse().toString()` para devolverlo en el orden correcto.

```
int i = r.length();
int j = t.length();

StringBuilder str = new StringBuilder("");

while (i > 0 && j > 0) {
    if (Utils.charEn(r,i) == Utils.charEn(t,j)) {
        str.append(Utils.charEn(r,i));
        i--;
        j--;
    } else if (m[i - 1][j] < m[i][j - 1]){
        str.append(Utils.charEn(r,i));
        i--;
    } else {
        str.append(Utils.charEn(t,j));
        j--;
    }
}

while (j > 0) {
    str.append(Utils.charEn(t,j));
    j--;
}

while (i > 0) {
    str.append(Utils.charEn(r,i));
    i--;
}

return str.reverse().toString();
}
```

Aunque haya elegido esta opción para resolverlo, he implementado también de diferentes maneras de devolver la secuencia correcta, comprobando la complejidad de cada solución.

Otro enfoque distinto:

```

public enum Dirección {
    DIAGONAL, ARRIBA, IZQUIERDA;
}

public String unaSolucionPD(){
    rellenarTabla();
    return reconstruirCadena();
}

public void rellenarTabla(){
    direcciones[0][0]=Dirección.ARRIBA;

    for (int i = 1; i < m.length; i++) {
        direcciones[i][0]=Dirección.ARRIBA;
    }

    for (int j = 1; j < m[0].length; j++) {
        direcciones[0][j]=Dirección.IZQUIERDA;
    }

    for (int i = 1; i < m.length; i++) {
        for (int j = 1; j < m[i].length; j++) {
            if(Utils.charEn(r,i) == Utils.charEn(t,j)) {
                direcciones[i][j] = Dirección.DIAGONAL;
            } else {
                if(m[i-1][j] < m[i][j-1]) {
                    direcciones[i][j] = Dirección.ARRIBA;
                } else {
                    direcciones[i][j] = Dirección.IZQUIERDA;
                }
            }
        }
    }
}

public String reconstruirCadena(){
    int i = r.length();
    int j = t.length();
    return reconstruyeRekursivamente(r,i,t,j);
}

public String reconstruyeRekursivamente(String r, int i, String t , int j){
    StringBuilder res = new StringBuilder("");

    if (i==0 && j==0) {
        return res.toString();
    }else if (j==0){
        return res.append(reconstruyeRekursivamente(r,i-1,t,j)).append(r.charAt(i-1)).toString();
    }else if (i==0){
        return res.append(reconstruyeRekursivamente(r,i,t,j-1)).append(t.charAt(j-1)).toString();
    } else {
        SCMC.Dirección flecha = direcciones[i][j];
        if (flecha.equals(Dirección.DIAGONAL)){

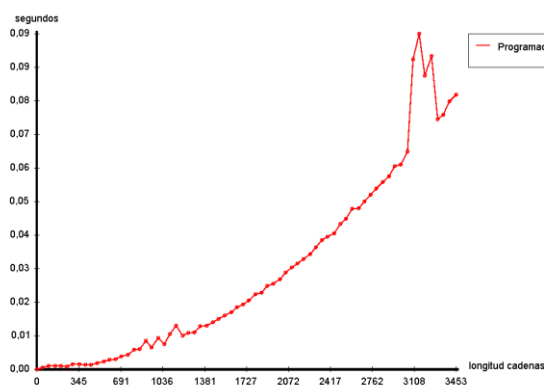
```

```

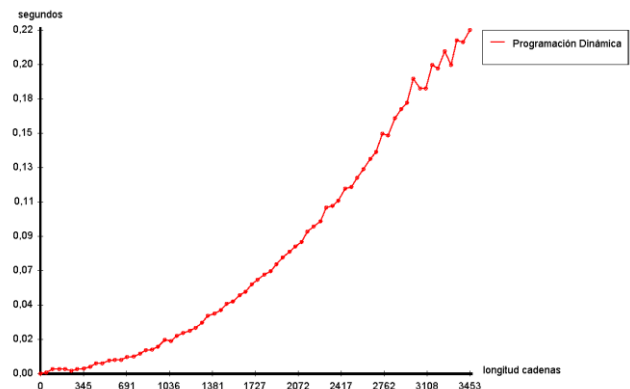
        return res.append(reconstruyeRekursivamente(r,i-
1,t,j-1)).append(r.charAt(i-1)).toString();
    }else if (flecha.equals(Dirección.ARRIBA)){
        return res.append(reconstruyeRekursivamente(r,i-
1,t,j)).append(r.charAt(i-1)).toString();
    } else {
        return res.append(reconstruyeRekursivamente(r,i,t,j-
1)).append(t.charAt(j-1)).toString();
    }
}
}
}

```

En este nuevo método `unaSolucionPD()`, construiremos una nueva matriz de tipo enumerado para acceder a los datos desde abajo, arriba y en diagonal, así como crearemos un método recursivo que irá creando la cadena desde las posiciones de esta nueva matriz. Seguiremos usando `StringBuilder` para ello. En un estudio de complejidad realizado, la primera forma es más eficiente en cuanto a la longitud de las cadenas con las que trabajamos, veamos una gráfica comparativa gracias a `TestsTiempos.java`:

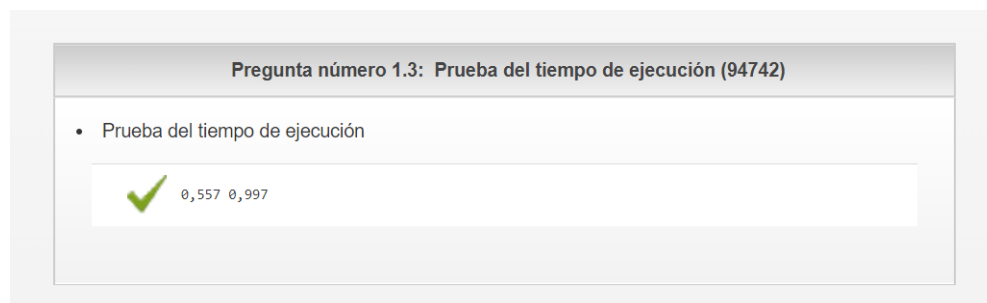


1º Método



2º Método

Debo también comentar que en esta práctica, más que el hecho de conseguir la solución óptima y pasar los tests bien, me he dedicado mucho a la eficiencia de la solución, en la corrección en *Siette*, he tenido que conseguir llegar a una eficiencia muy alta:



Por ello, considero que la eficiencia de mi solución es bastante considerable.